Jayden Tran, Joseph Gereige, Tianhong Zhong

Dr. Fatema

EECE 2140: Computing Fundamentals for Engineers

30 October 2026

Circuit Solver: Iteration 2 System Design and Implementation

Team 02

GitHub Repository: https://github.com/zhongtia111/circuit_solver_2140

1. Project Leader

The Project Leader for Iteration 2 is Tianhong Zhong, who had not served as Project Leader before. Along with owning input parsing and the shared data structures, Tianhong coordinates daily progress, makes sure tasks are shared fairly, keeps the task table up to date, and presents the status report at the check-in meeting.

Table 1. Project Leader Responsibilities

Responsibility	**How It Is Carried Out**
Coordinate daily progress	Checks in with each member through the team group chat on what they finished and what they are working on next.
Share tasks fairly	Reviews the task table so each member owns a comparable share of the work, and reassigns tasks at the weekly meeting if someone falls behind.
Keep the task table updated	Updates task status in the Excel project plan as tasks move from Not started to Stub, In progress, and Done.
Lead the menu framework	Builds the menu and main loop (T9) first, so every member has a compiling program to add their code to.
Present the status report	Presents the check-in status report in Section 7.

2. Task Definitions

Every feature of Circuit Solver is listed below as a separate task. Tasks T1 through T7 each run from one menu option. Tasks T8 through T10 support the whole program and do not have their own menu option: T8 is the shared data structures that every task uses, T9 is the menu itself, and T10 is the test circuits used to verify the results. The status key is Not started, Stub, In progress, and Done.

Table 2. Task Assignments

ID	Task	Menu Option	Owner	Status
T1	Load netlist from file	1	Tianhong Zhong	Not started
T2	Enter netlist from terminal	2	Tianhong Zhong	Not started
T3	Display current circuit	3	Tianhong Zhong	Not started
T4	Validate circuit	4	Jayden Tran	Not started
T5	Solve circuit	5	Joseph Gereige	Not started
T6A	Compute branch currents and power	6	Joseph Gereige	Not started
T6B	Display results	6	Jayden Tran	Not started
T7	Load example circuit	7	Jayden Tran	Not started
T8	Shared data structures	None (core)	Tianhong Zhong	Not started
T9	Menu and main loop	None (framework)	Tianhong Zhong	Not started
T10	Test circuits and verification	None (testing)	Jayden Tran	Not started
T11	README and final documentation	None (docs)	Joseph Gereige	Not started

Table 3. Files, Functions, and Deliverables

ID	Files and Functions	Expected Deliverable
T1	parser.h/.cpp: loadFromFile(), parseLine()	Reads a .txt netlist into the Circuit object.
T2	parser.h/.cpp: readFromTerminal(), parseLine()	Builds a Circuit from netlist lines typed into the terminal.
T3	circuit.h/.cpp: Circuit::print()	Prints the loaded components and nodes.
T4	validate.h/.cpp: validateCircuit(), hasGroundNode(), checkValues()	Flags a missing ground node, negative or zero values, and bad formatting with clear messages.
T5	solver.h/.cpp: buildNodalMatrix(), gaussianElimination(), isSingular()	Node voltages within 0.1% of hand calculations; an error message for singular or unsolvable circuits.
T6	output.h/.cpp: printResults(), printNodeVoltages(), printBranchCurrents(), printPower()	Formatted table of the voltage, current, and power for every component.
T7	parser.h/.cpp (reuses loadFromFile()) tests/ example files	Loads a built-in test circuit for a quick demonstration.
T8	circuit.h/.cpp: class Component, class Circuit, addComponent()	Component and Circuit classes that every other module builds on.
T9	main.cpp: showMenu(), switch dispatch	Program compiles and runs with every menu option connected to a stub.
T10	tests/: at least 10 netlists with expected answers	At least 10 hand-solved circuits committed; solver output matches within 0.1%.

3. System Design

3.1 Classes

The program is built around six classes. Component stores one circuit element, and Circuit holds the list of components that make up the netlist. Parser fills a Circuit from a file or the terminal, Validator checks it, Solver turns it into the nodal analysis matrix and solves for the node voltages, and Output uses those voltages to print the results. Because every module works

through the shared Circuit class, each member can build their part at the same time once T8 is in place.

Table 4. Classes

Class	Private Data	Public Methods	Owner
Component	name (string) type (char: R, V, or I) nodeA, nodeB (int) value (double)	getName(), getType(), getNodeA(), getNodeB(), getValue() print()	Tianhong Zhong
Circuit	components (vector<Component>) numNodes (int)	addComponent() getComponents(), getNodeCount() print()	Tianhong Zhong
Parser	lineNumber (int) lastError (string)	loadFromFile() readFromTerminal() parseLine()	Tianhong Zhong
Validator	errors (vector<string>)	validateCircuit() hasGroundNode() checkValues()	Jayden Tran
Solver	matrix (vector<vector<double>>) rhs (vector<double>) nodeVoltages (vector<double>) branchCurrents (vector<double>) powers (vector<double>)	buildNodalMatrix() gaussianElimination() isSingular() getNodeVoltages() computeBranchCurrents() computePower() getNodeVoltages() getBranchCurrents() getPowers()	Joseph Gereige
Output	precision (int)	printResults() printNodeVoltages() printBranchCurrents() printPower()	Jayden Tran

3.2 Menu Design

The program shows a text menu in a loop. The user types a number, and a switch statement in main.cpp calls exactly one task function for that option. Any number outside 0 to 7, or any input that is not a number, prints an error message and shows the menu again.

Table 5. Menu Options

Option	Menu Text	Task
1	Load netlist from file	T1
2	Enter netlist from terminal	T2
3	Display current circuit	T3
4	Validate circuit	T4
5	Solve circuit	T5
6	Display results	T6
7	Load example circuit	T7
0	Exit	Ends the main loop

3.3 File Structure

Each class has its own header and source file, and each file has a single owner. Members only edit the files they own, which keeps merge conflicts to a minimum.

Table 6. File Structure

File	Contents	Owner
main.cpp	showMenu() and the switch statement (T9)	Tianhong Zhong
circuit.h / circuit.cpp	Component and Circuit classes (T3, T8)	Tianhong Zhong
parser.h / parser.cpp	Parser class (T1, T2, T7)	Tianhong Zhong
validate.h / validate.cpp	Validator class (T4)	Jayden Tran
solver.h / solver.cpp	Solver class (T5)	Joseph Gereige
output.h / output.cpp	Output class (T6)	Jayden Tran
tests/	Example circuit and test netlists with expected answers (T7, T10)	Jayden Tran
docs/	This report (PDF) and the Excel project plan	All
README.md	Build and run instructions	Tianhong Zhong

T7 reuses Parser::loadFromFile() to read an example netlist from the tests/ folder, so it does not need a separate file. Jayden owns the example files and the menu task, and Tianhong owns the parser code they rely on.

3.4 Class Diagram

Figure 1 shows how the classes connect. main.cpp calls Parser, Validator, Solver, and Output through the menu. All four work on one shared Circuit object, which contains one or more Component objects. Output also gets the solved node voltages from Solver so it can calculate branch currents and power.

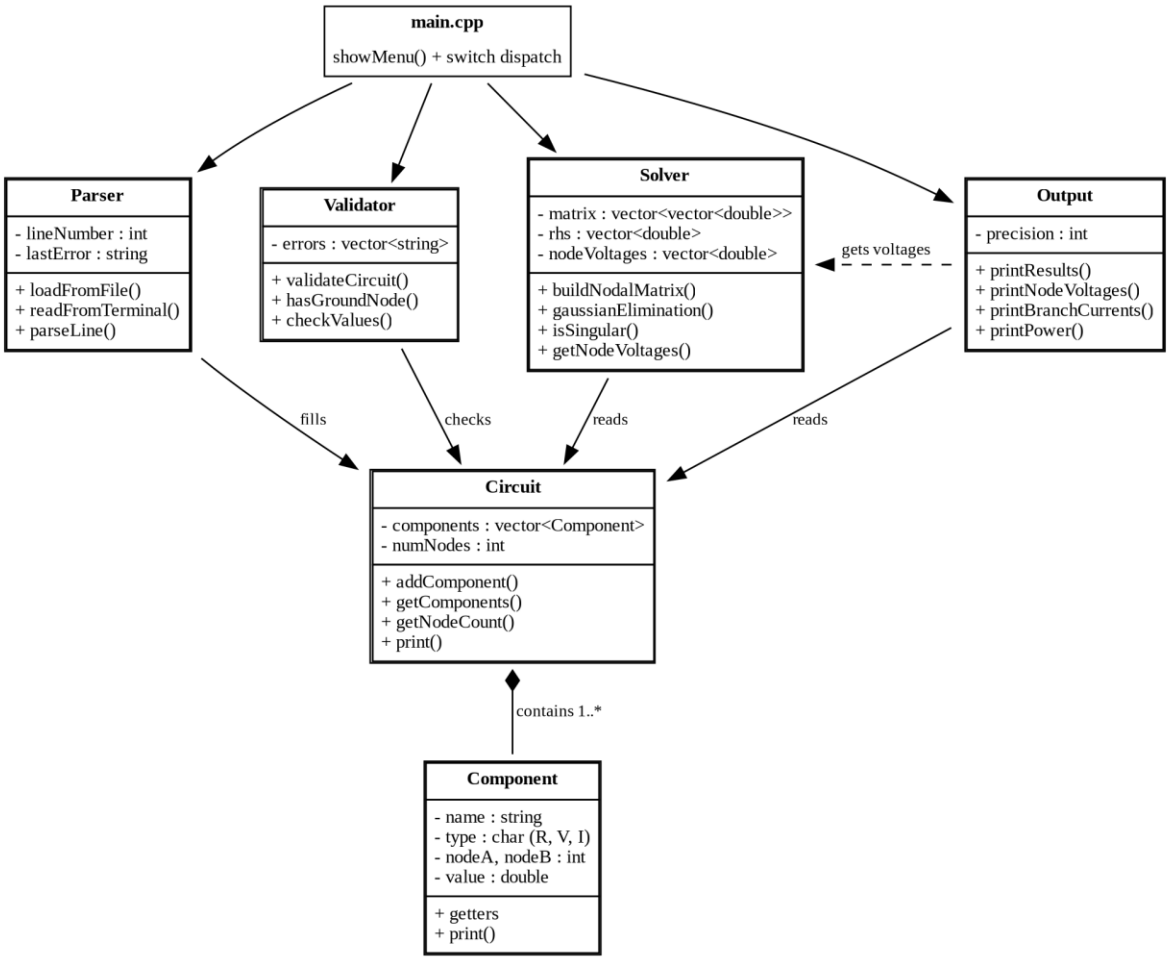


Fig. 1. Circuit Solver class diagram.

4. Implementation

4.1 Implementation Plan

The menu and main loop are built first. Every task function starts as a stub that prints "not implemented yet," so the program compiles and runs from the first day with every menu option connected. The program is built with g++ -std=c++17 -Wall -Wextra -pedantic, matching the team goal of compiling with no warnings. Once the stubs are in place, each member replaces only their own stubs with working code and does not edit tasks owned by other members.

4.2 Team Workflow

Each member works on their own feature branch and opens a pull request into main, which another member reviews before it is merged. Work is committed to GitHub regularly with clear commit messages that name the task, for example "T4: add missing ground check." The commit history on GitHub serves as the record of each member's work.

5. Revised Timeline

Dates from the Iteration 1 plan are kept. New Iteration 2 tasks are marked with an asterisk (*); their dates are proposed and will be confirmed at the next team meeting.

Table 7. Revised Timeline

Milestone	Task	Owner	Deadline	Status
M1: Planning	Write project plan and set up GitHub repository	All	10/13	Done
M2: Design	Define circuit input format and data structures	Tianhong Zhong	10/18	Done
M2: Design	Choose solving method	Joseph Gereige	10/20	Done
M2: Design	Design classes, menu, and file structure *	Tianhong Zhong (PL)	10/25 *	In progress

Milestone	Task	Owner	Deadline	Status
M3: Build	Menu and main loop with all stubs *	Tianhong Zhong (PL)	10/27 *	Not started
M3: Build	Shared data structures (Component, Circuit) *	Tianhong Zhong	10/30 *	Not started
M3: Build	Input parsing (file and terminal)	Tianhong Zhong	11/03	Not started
M3: Build	Input validation *	Jayden Tran	11/05 *	Not started
M3: Build	Matrix setup and solver	Joseph Gereige	11/10	Not started
M3: Build	Branch current and power calculation	Joseph Gereige	11/10	Not started
M3: Build	Output and results display	Jayden Tran	11/10	Not started
M4: Testing	Create test circuits and verify results	Jayden Tran	11/17	Not started
M5: Final	README and final documentation	Joseph Gereige	11/24	Not started
M5: Final	Final presentation and live demo	All	12/08	Not started

5.1 Changes from Iteration 1

Input parsing moved from Jayden to Tianhong, so the parser is built by the same person who designs the data structures it fills. Input validation was made its own task and assigned to Jayden, which removes the overlap in the Iteration 1 plan, where validation appeared in both Jayden's and Tianhong's objectives. Jayden keeps output display and testing and adds the example-circuit loader. Four tasks were added to the timeline for this iteration: the system design, the menu with stubs, the shared data structures, and input validation.

6. Contribution Table

Table 8. Member Contributions

Team Member	Role / Focus	This Iteration (Owns)	Next Steps
Tianhong Zhong	Project Leader and Input Parsing	Define all tasks and the class design; build the menu and main loop with stubs (T9); implement the shared data structures Component and Circuit (T8); build the netlist parser for file and terminal input (T1, T2); display the current circuit (T3); coordinate progress and keep the task table current.	Finalize the class diagram and file layout; get the menu compiling with all stubs; replace the parser stubs with working code; draft the README once the input format is set.
Jayden Tran	Validation, Output, and Testing	Build input validation for a missing ground node, negative or zero values, and bad formatting (T4); implement the results display for voltages, currents, and power (T6); build the example-circuit loader (T7); create and verify at least 10 hand-solved test circuits (T10).	Replace the validation and output stubs with working code; commit the first set of hand-solved test circuits with expected answers.
Joseph Gereige	Solver Engine	Write the method summary for modified nodal analysis and Gaussian elimination; build the solver engine with matrix setup and Gaussian elimination (T5); compute the branch currents and power for every component (T6a); detect singular or unsolvable circuits and report an error; own the README and final documentation (T11).	Replace the solver stub with working code; add the branch-current and power calculations; confirm node voltages, currents, and power match hand calculations within 0.1% on the test circuits; draft the README once the input format is set.

7. Check-In Status Report

Presented by Tianhong Zhong, Project Leader, at the check-in meeting on 10/29.

Table 9. Status Report

Item	Summary
Completed	Iteration 1 project plan and GitHub repository (M1). All ten tasks defined with an owner, menu option, files and functions, and deliverable.
In progress	Input format and data structures (Tianhong); method summary for nodal analysis (Joseph); class, menu, and file structure design (Tianhong).
Next steps	Build the menu with all stubs so the program compiles and runs (Tianhong); implement Component and Circuit (Tianhong); each member then replaces their own stubs, starting with parsing (Tianhong), validation (Jayden), and the solver (Joseph).
Open items	Confirm the proposed Iteration 2 dates marked with * at the next team meeting.

8. Team Meetings and Decisions

Table 10. Meeting Log

Date	Attendees	Topics Discussed	Decisions Made	Action Items
10/13	Jayden Tran, Tianhong Zhong, Joseph Gereige	Iteration 1 planning	Chose roles, goals, and deadlines	See Iteration 1 plan
[MM/DD]	[Names]	Iteration 2 system design and task assignments	Tianhong as Project Leader; parsing moved to Tianhong; validation assigned to Jayden	Menu with stubs (Tianhong); proposed dates to be confirmed

8.1 Summary of Key Decisions

For Iteration 2, the team chose Tianhong Zhong as Project Leader, since Tianhong had not led before and is already building the data structures and menu that the rest of the program depends on. The team split the project into ten tasks, one for each menu option plus three supporting tasks for the data structures, the menu, and testing, so every feature has a single owner and a clear deliverable. Input parsing was moved to Tianhong and input validation became Jayden's task, which removed the overlap in the first plan and keeps the parser with the

person defining the format it reads. The program is organized so each class has its own header and source file with a single owner, which lets all three members work at the same time without editing each other's code. Finally, the team agreed to build the menu with stubs first, so the program compiles and runs from the start and each member can replace their own stubs as their code is finished.